
Module 06: Forms & Server Actions

Build full-stack forms with server-side validation, progressive enhancement, and secure data handling.

Revolution Trading Pros - Web Dev Course

Overview

Forms are the backbone of interactive web applications. SvelteKit provides a powerful form handling system that works with progressive enhancement - your forms work without JavaScript, then get enhanced when JS is available. In this module, you will learn how to build secure, validated forms with server actions.

HTML Forms in SvelteKit

SvelteKit forms use standard HTML form elements. The key difference is that form submissions are handled by server-side actions, not client-side JavaScript.

```
<!-- src/routes/login/+page.svelte -->
<script lang="ts">
  let { form } = $props();
</script>

{#if form?.error}
  <p class="error">{form.error}</p>
{/if}

<form method="POST">
  <label>
    Email
    <input type="email" name="email" value={form?.email ?? ""} required />
  </label>

  <label>
    Password
    <input type="password" name="password" required />
  </label>

  <button type="submit">Log In</button>
</form>

<style>
  .error { color: #ff3e00; padding: 0.5rem; background: #fff0f0; }
  form { display: flex; flex-direction: column; gap: 1rem; max-width: 400px; }
  label { display: flex; flex-direction: column; gap: 0.25rem; }
  input { padding: 0.5rem; border: 1px solid #ccc; border-radius: 4px; }
</style>
```

Notice that we use standard HTML: `method="POST"`, name attributes on inputs, and a submit button. The form's action attribute defaults to the current page URL. SvelteKit intercepts the submission and routes it to the server action.

| Form Actions (+page.server.ts)

Form actions are server-side functions that handle form submissions. They are defined in +page.server.ts alongside the load function.

Default Action

```
// src/routes/login/+page.server.ts
import type { Actions } from ".$types";
import { fail, redirect } from "@sveltejs/kit";

export const actions: Actions = {
  default: async ({ request, cookies }) => {
    const data = await request.formData();
    const email = data.get("email") as string;
    const password = data.get("password") as string;

    // Validation
    if (!email || !email.includes("@")) {
      return fail(400, {
        error: "Please enter a valid email address.",
        email // Return email so the form can repopulate it
      });
    }

    if (!password || password.length < 8) {
      return fail(400, {
        error: "Password must be at least 8 characters.",
        email
      });
    }

    // Authentication logic
    const user = await authenticateUser(email, password);

    if (!user) {
      return fail(401, {
        error: "Invalid email or password.",
        email
      });
    }

    // Set session cookie
    cookies.set("session_id", user.sessionId, {
      path: "/",
      httpOnly: true,
      sameSite: "lax",
      secure: true,
      maxAge: 60 * 60 * 24 * 7 // 1 week
    });

    // Redirect to dashboard
    redirect(303, "/dashboard");
  }
}
```

};

Named Actions

You can define multiple named actions for a single page. This is useful when a page has multiple forms.

TypeScript

```
// src/routes/todos/+page.server.ts
import type { Actions, PageServerLoad } from "./$types";
import { fail } from "@sveltejs/kit";

export const load: PageServerLoad = async () => {
  const todos = await getTodos();
  return { todos };
};

export const actions: Actions = {
  create: async ({ request }) => {
    const data = await request.formData();
    const text = data.get("text") as string;

    if (!text?.trim()) {
      return fail(400, { error: "Todo text is required" });
    }

    await createTodo(text.trim());
    return { success: true };
  },

  delete: async ({ request }) => {
    const data = await request.formData();
    const id = data.get("id") as string;
    await deleteTodo(id);
    return { success: true };
  },

  toggle: async ({ request }) => {
    const data = await request.formData();
    const id = data.get("id") as string;
    await toggleTodo(id);
    return { success: true };
  }
};
```

```
<!-- src/routes/todos/+page.svelte -->
<!-- Use ?/ prefix for named actions -->
<form method="POST" action="?/create">
  <input type="text" name="text" placeholder="Add a todo..." />
  <button>Add</button>
</form>

{#each data.todos as todo}
  <div>
    <form method="POST" action="?/toggle">
      <input type="hidden" name="id" value={todo.id} />
      <button>{todo.done ? "Undo" : "Done"}</button>
    </form>
    <span>{todo.text}</span>
    <form method="POST" action="?/delete">
      <input type="hidden" name="id" value={todo.id} />
      <button>Delete</button>
    </form>
  </div>
{/each}
```

Validation with fail()

The `fail()` function returns validation errors to the client without redirecting. The data you return is available as the `'form'` prop in the page component.

```
// Comprehensive validation example
export const actions: Actions = {
  register: async ({ request }) => {
    const data = await request.formData();
    const name = data.get("name") as string;
    const email = data.get("email") as string;
    const password = data.get("password") as string;
    const confirm = data.get("confirm") as string;

    const errors: Record<string, string> = {};

    if (!name || name.length < 2) {
      errors.name = "Name must be at least 2 characters";
    }

    if (!email || !email.includes("@")) {
      errors.email = "Valid email is required";
    }

    if (!password || password.length < 8) {
      errors.password = "Password must be at least 8 characters";
    }

    if (password !== confirm) {
      errors.confirm = "Passwords do not match";
    }

    if (Object.keys(errors).length > 0) {
      return fail(400, {
        errors,
        name,
        email // Do NOT return password
      });
    }

    await createUser(name, email, password);
    redirect(303, "/login?registered=true");
  }
};
```

```
<!-- Display field-level errors -->
<script lang="ts">
  let { form } = $props();
</script>

<form method="POST" action="?/register">
  <label>
    Name
    <input name="name" value={form?.name ?? ""} />
    {#if form?.errors?.name}
      <span class="error">{form.errors.name}</span>
    {/if}
  </label>

  <label>
    Email
    <input type="email" name="email" value={form?.email ?? ""} />
    {#if form?.errors?.email}
      <span class="error">{form.errors.email}</span>
    {/if}
  </label>

  <!-- Password fields... -->
  <button>Register</button>
</form>
```

Progressive Enhancement with `use:enhance`

The `use:enhance` directive progressively enhances your forms with JavaScript. Without it, forms work as standard HTML forms (full page reload). With it, submissions are handled via `fetch` with no page reload.

```

<script lang="ts">
  import { enhance } from "$app/forms";

  let { form } = $props();
  let submitting = $state(false);
</script>

<!-- Basic enhancement -->
<form method="POST" use:enhance>
  <!-- Form automatically submits via fetch, no page reload -->
</form>

<!-- Custom enhancement with callbacks -->
<form
  method="POST"
  action="?/create"
  use:enhance={() => {
    submitting = true;

    return async ({ update, result }) => {
      submitting = false;

      if (result.type === "success") {
        // Custom success handling
        console.log("Todo created!");
      }

      await update(); // Apply the default behavior
    };
  }}
>
  <input type="text" name="text" />
  <button disabled={submitting}>
    {submitting ? "Saving..." : "Add Todo"}
  </button>
</form>

```

The enhance callback receives the form and cancel function. The return function receives the result and an update function. The result.type can be 'success', 'failure', 'redirect', or 'error'.

Redirects

Use the `redirect()` function from `@sveltejs/kit` to redirect after successful form submissions or in load functions.

TypeScript

```
import { redirect } from "@sveltejs/kit";

// In a form action (after successful submission)
redirect(303, "/dashboard");

// In a load function (e.g., auth guard)
export const load = async ({ locals }) => {
  if (!locals.user) {
    redirect(307, "/login");
  }
  return { user: locals.user };
};
```

Status codes matter: Use 303 for POST-redirect-GET (after form submission). Use 307 for temporary redirects that preserve the HTTP method. Use 308 for permanent redirects.

Module 6 Exercise

Exercise: Contact Form with Validation

Put what you have learned into practice with this hands-on exercise.

- %I Step 1: Create a /contact route with a +page.svelte and +page.server.ts.
- %I Step 2: Build a form with fields: name, email, subject (dropdown), and message (textarea).
- %I Step 3: Implement server-side validation in the form action: all fields required, email must be valid, message must be at least 20 characters.
- %I Step 4: Return field-level errors using fail() and display them next to each field.
- %I Step 5: Repopulate the form with submitted values on validation failure.
- %I Step 6: Show a success message when the form submits successfully.
- %I Step 7: Add use:enhance for progressive enhancement with a loading state.
- %I Step 8: Style the form with proper spacing, focus states, and error styling.
- %I Step 9: Test that the form works both with and without JavaScript.
- %I Step 10: Commit your changes to Git.

Summary

- %I SvelteKit forms use standard HTML with method="POST" and name attributes.
- %I Form actions in +page.server.ts handle submissions on the server.
- %I Named actions (?/actionName) allow multiple forms per page.
- %I fail() returns validation errors without redirecting.
- %I use:enhance adds progressive enhancement with fetch-based submissions.
- %I redirect() navigates after successful submissions with proper status codes.